biohub-cell-tracking-during-development

3rd Place Solution

Rank	3
Team	yu4u
Author	yu4u
Topic ID	744484
Version	Original

Source: https://www.kaggle.com/competitions/biohub-cell-tracking-during-development/discussion/744484
Retrieved with Kaggle CLI on 2026-10-04.

Thank you to the organizers and to everyone who shared discussions and notebooks. Our solution focused on detecting cells from sparse annotations and combining motion and division evidence into a consistent lineage graph.

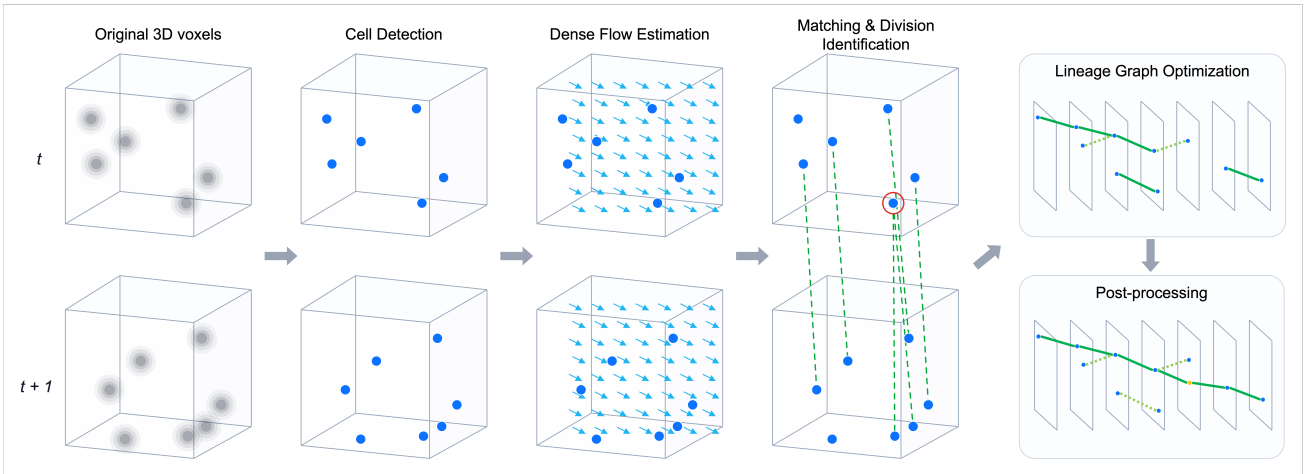
1. Overview

Our pipeline has six stages:

1. **Cell detection:** ensemble heatmaps from 2.5D U-Nets and a 3D SegResNet to detect cell centers.
2. **Dense flow:** estimate a 3D displacement field between consecutive frames.
3. **Cell matching:** score candidate correspondences using image features and positions corrected for motion.
4. **Division identification:** identify dividing parents from original and motion-aligned images, with auxiliary models for the stages before and after division.
5. **Lineage graph optimization:** jointly select ordinary links and division events.
6. **Post-processing:** fill short gaps, remove small components, and refine node coordinates.

We put particular effort into detection accuracy because every downstream stage depends on the detected cells. For tracking, we trained the flow, matching, and division models separately, then combined their outputs through graph optimization to determine the connections between cells. Our final submission used an ensemble of relatively large detection backbones, and detection accounted for 55% of the total runtime.

Our final solution achieved a CV score of **0.977801 (0.540107)**, a public leaderboard score of **0.977 (0.52)**, and a private leaderboard score of **0.967 (0.47)**. Values in parentheses indicate Division Jaccard.



2. Data and Validation

The training set contains 199 videos cropped from two embryos. Each video has 100 frames of 64 × 256 × 256 voxels. Voxel spacing is 1.625 µm along z and 0.40625 µm along x and y, so we compute distances in physical coordinates.

The main challenge is that the annotations cover only a small fraction of the cells. There are approximately 133,000 annotated nodes, about 2.8% of the total estimated by the organizers. Many clearly visible cells have no annotation, and the training set contains only 151 annotated divisions.

This sparsity also matters for evaluation. The official score is the sum of the node-count-adjusted edge Jaccard and 0.1 times the division Jaccard. Links entirely within unannotated regions are not automatically false positives; incorrect links that compete with annotated connections are penalized. A separate adjustment accounts for the number of predicted nodes. Detection precision or local link classification accuracy alone therefore does not describe the final score.

We used five-fold cross-validation at the video level, distributing videos from both embryos across the folds and keeping all frames of a video together. We generated out-of-fold (OOF) predictions for the detectors, matcher, and division models. The downstream calibration models were also trained with the evaluation fold excluded.

Both test embryos are different from the training embryos. We considered training on one embryo and validating on the other, but used video-level five-fold CV for model selection based on the improvement trends observed in CV and on the public leaderboard. Crops from the same embryo can overlap in space and time across folds, so this validation is not fully independent. Agreement with the public leaderboard also does not establish generalization to the private test embryo.

The four embryos are summarized below. Training prefixes and video counts come from the provided data; test prefixes, counts, density ranges, and public/private split assignments were identified through probing.

Embryo prefix	Videos	Split	Density p10	Density p50 (median)	Density p90
`44b6`	71	Train	134	355	627
`6bba`	128	Train	54	114	445
`fdad`	60	Public test	[40, 60)	[160, 260)	[260, 420)
`ea36`	106	Private test	[40, 60)	[60, 100)	[160, 260)

Here, density means **detections per frame**, used as a proxy for the number of cells. Using the same early version of our detector for all videos, we counted detections in four evenly spaced frames per video, averaged the counts, and computed percentiles across videos. Training values are rounded measurements, while test values are intervals inferred from probing. The public and private test embryos differ substantially in cell density.

3. Cell Detection

Backbones

We combined 2.5D and fully 3D detectors.

For the 2.5D models, we reused the backbone architecture from our CZII - CryoET Object Identification solution. A pretrained 2D encoder extracts features from z slices, features are pooled along z at each stage, and a 3D decoder combines them. We used EfficientNetV2-L and EfficientNet-B7 encoders.

Each encoder input stacks three neighboring z slices as channels, allowing the first layer to use local depth information. These slices provide spatial context only; temporal information is introduced later in the tracking pipeline.

The fully 3D detector uses MONAI's SegResNet implementation, a residual encoder-decoder built from 3D convolutions. We used 32 initial channels, four encoder stages with one residual block per stage, and Group Normalization.

The submission used the following ensemble:

Detector	Architecture	Heatmap weight
EfficientNetV2-L	2.5D U-Net	0.3
EfficientNet-B7	2.5D U-Net	0.4
SegResNet (MONAI)	3D residual encoder-decoder	0.3

Training with sparse annotations

The target heatmap places a Gaussian with a standard deviation of 2 µm at each annotated center. Applying MSE directly to this target would also train the detector to predict zero at unannotated cells.

To avoid treating these cells as negatives, we extracted candidate centers using Difference of Gaussians (DoG) with a low threshold. We excluded regions within 6 µm of unmatched candidates from the loss. DoG candidates within 2 µm of an annotated center were treated as duplicates and did not create exclusion regions. Positive regions around annotated centers remained supervised even when they overlapped an exclusion region. DoG thus identified uncertain background regions to mask, without adding positive pseudo-labels.

We averaged MSE separately over positive and background regions, with a background weight of 0.5. This prevents the much larger background volume from overwhelming the supervision around cell centers.

Image intensities were normalized using the 0.1st and 99.9th percentiles of each video. Training crops were 48 × 192 × 192 voxels. Augmentations included flips and rotations in the xy plane, intensity changes, and blur.

Scaling and rotation augmentations degraded performance in our experiments.

Inference

For each detector, we averaged heatmaps from the five fold models and used two test-time views: the original image and an image flipped along both x and y. We combined the detector heatmaps with the weights above, then extracted centers using non-maximum suppression with a `(3, 7, 7)` kernel and a threshold of 0.2.

A0: Detection with geometric linking

The baseline in Table 1 links detections in consecutive frames using physical distance and a one-to-one Hungarian assignment with a 7 µm distance gate. We estimate collective translation, or drift, from the median displacement of matched points and repeat drift correction and assignment twice.

This baseline uses no dense flow, learned matcher, division predictions, or post-processing. Simple linking provides a starting point for evaluation: detections alone cannot produce true positive edges, while ordinary links can earn an edge score without predicting divisions.

4. Dense Flow and Cell Matching

A1: Dense flow

Cells move both individually and collectively. We estimate flow from consecutive 3D images to predict where each cell will move in the next frame.

Downsampling by `(z, y, x) = (1, 4, 4)` gives an isotropic grid with 1.625 µm spacing. A small 3D encoder-decoder operates on this grid. Local correlations and a soft-argmax produce an initial displacement, which the network refines with a residual prediction. The encoder uses 32, 64, and 128 channels without normalization layers.

On real image pairs, training combines image and local feature consistency after warping, forward-backward consistency, and smoothness losses. We also directly supervise displacement on synthetic pairs generated with known translations and smooth deformations. Annotated links are used for endpoint-error evaluation and checkpoint selection, but not in the training loss.

The estimated flow supports both correspondence search and alignment of the images supplied to the division model.

In A1, we add dense flow to the source coordinates, estimate residual drift twice, and retain the same 7 µm gate and one-to-one assignment as A0. This measures the effect of motion compensation before introducing division modeling.

A2: Cell matching

The matcher takes two consecutive frames and their detections, and scores which cells correspond across time. It uses appearance and positions corrected by flow to predict candidate correspondence scores and a null score representing the absence of a match. These scores narrow the candidate set; graph optimization later selects the final connections.

The matcher has its own 2.5D U-Net with an EfficientNetV2-S encoder, separate from the detectors. It uses the same `(1, 4, 4)` downsampling as the flow model. We sample 64-channel features at each detection from an intermediate decoder stage, avoiding computation of the full-resolution decoder output.

Self-attention and cross-attention process cell features and positional information. The matching head refines scores based on appearance cosine similarity and uses role embeddings to distinguish the two frames. It also predicts the null option.

For training, detections matched to annotated cells act as anchors, and annotated edges provide targets for bidirectional cross-entropy. Unannotated cells remain competing candidates, but absence of annotation does not make them null targets. For a dividing parent, we construct a target for each daughter and exclude the other daughter from the negative set.

For ordinary links, we retain candidates among the matcher's top five probabilities whose distance after flow compensation is at most 12 µm. We pass their ranks, forward and reverse probabilities, and margins over competing candidates to the next stage.

In A2, we use these candidates and scores in a Hungarian assignment. A0 and A1 consider all pairs within 7 µm after drift correction; A2 applies the same gate to the matcher candidates. The A1-to-A2 change therefore

includes candidate filtering. In the complete pipeline, joint graph optimization replaces this local assignment and uses candidates within the 12 µm flow distance gate.

5. Division Identification (A3)

The division model predicts whether a detected cell will split into two daughters in the next frame, using 3D images from neighboring time points. We call this the **parent model**. Its output is combined with matching scores and daughter geometry, and graph optimization chooses the parent-to-daughter connections.

Both division and ordinary motion change the appearance of a local image region. To help distinguish them, we align the neighboring frames to the central frame before passing them to the model. We also retain the original images because a single displacement field cannot fully represent a one-to-two split.

The main parent model receives five volumes from three time points:

```
[original t−1, t−1 aligned to t, t, t+1 aligned to t, original t+1]
```

The division model uses `(1, 2, 2)` downsampling. When converting flow to this grid, we account for both displacement units and voxel-center offsets introduced by pooling. We sample the next frame using forward flow. For the previous frame, we approximate the inverse displacement by negating the preceding forward flow field.

A shared EfficientNetV2-S encoder processes each input. Features from corresponding stages are concatenated in input order and passed to a 3D decoder with Group Normalization. The decoder predicts a parent score at each detection. The flow model remains frozen.

Positive examples are annotated parents with two children in the next frame. Negatives include ordinary continuing cells and daughters immediately after division. Detections without a GT match do not enter the negative loss. We average positive and negative BCE separately. During training, 25% of samples come from frames containing positive parents, 25% from hard-negative frames around divisions, and the remainder from ordinary sampling. We use AdamW with a learning rate of 1e-4, 20 epochs, and EMA decay of 0.995.

We also use independent **pre** and **post** models. For a division with the parent at time t and daughters at t+1, pre identifies the precursor at t−1, while post identifies the daughters at t+1. Each model receives three original frames centered on the cell's evaluation time. Their scores provide additional evidence for division candidates.

The submission uses the parent model with both original and aligned images to build the primary division candidate pool. A second parent model takes only three original frames. Its scores, together with pre and post scores, support ordinary-link calibration and an additional division candidate pool with a 16 µm radius. The two division pools are calibrated separately. Division inference averages five fold models and four rotations in the xy plane; OOF evaluation uses the corresponding held-out model for each video.

6. Lineage Graph Optimization (A3)

Calibrating candidate scores

We combine distances and model outputs to estimate probabilities for ordinary links and division candidates. These probabilities let the graph optimizer compare competing choices.

Candidates are either ordinary links or division events consisting of one parent and two daughters. A parent score threshold of 0.1 controls division candidate generation; final selection is handled by the optimizer.

Calibration features include distance, bidirectional matching scores, candidate ranks, margins, detection confidence, and parent/pre/post scores. We aggregate pre evidence over several possible predecessors. Examples include the maximum of the pre score multiplied by the matching probability, and an average weighted by matching probabilities.

To handle sparse annotations, we estimate two probabilities:

- **Probability of being evaluated:** whether a candidate is subject to scoring given the available annotations.
- **Conditional probability of being correct:** whether it is a true positive, given that it is evaluated.

Writing these as a and q, we use `a × q` as the expected true-positive contribution and `a × (1 − q)` as the expected false-positive contribution. This avoids treating all unevaluated candidates as errors during training.

Ordinary links use logistic regression. For the conditional division probability, we train LightGBM and XGBoost from a linear model's initial logit and average their predicted probabilities with equal weights. The primary and additional division pools have separate models. Features include aggregated predecessor evidence, motion, and daughter separation. We retain the LightGBM probabilities for constructing the initial graph and use the averaged probabilities when evaluating choices during optimization. Division calibration targets require exact matches to the parent and both daughters, which differs from the official metric's local lineage criterion.

Selecting links and divisions jointly

If ordinary links are fixed first, a correct daughter may be assigned to another parent, preventing a valid division from being added later. We therefore jointly choose which nodes to retain, which ordinary links to use, and which daughter pairs to select as divisions.

The main constraints are:

- Each cell has at most one parent.
- Each cell has at most one outgoing event: an ordinary link or a division.
- A division selects both daughters together.
- A dividing parent must have an ordinary incoming link, and each daughter must have an ordinary outgoing link.
- A daughter cannot divide again at the immediately following time boundary.

The last two constraints reflect the temporal resolution of this dataset. They also prevent implausible divisions from supporting one another and strengthen the LP relaxation.

The objective is a surrogate of the competition metric computed from expected true positives, false positives, and node counts. We linearize it around the current solution to obtain rewards for true positives and costs for false positives and nodes. We optimize each video using these coefficients, aggregate expected counts across videos, and update the coefficients for up to three rounds. We accept an updated solution only if it improves the surrogate objective.

We use HiGHS, first solving the linear programming (LP) relaxation. If fractional decisions remain, we solve a mixed-integer linear program (MILP) with binary division variables. If the time limit is reached, we retain a feasible solution. Optimization uses predicted probabilities; the resulting graphs are evaluated separately with the official scoring implementation.

7. Post-processing

A4: Gap closing

To recover temporarily missed cells, we connect track endpoints to the starts of later tracks. We estimate collective motion from selected links, compensate for it, and use Hungarian assignment to select connections.

We process gaps of one, two, and three missing frames in that order. For g missing frames, the distance gate is $3\sqrt{(g+1)}$ µm. We insert nodes by linearly interpolating between the original endpoint coordinates.

A5: Short component removal

After gap closing, we remove weakly connected components with fewer than six nodes. Gap closing comes first because a short fragment may become part of a longer track once missing detections are filled in.

A6: Affine refinement of detected coordinates

Even with correct connections, localization jitter can affect matching to GT points. We refine coordinates while holding the graph connections fixed.

For each pair of consecutive frames, we fit an affine model that predicts displacement from position using ordinary links. A quadratic optimization then balances consistency with the predicted motion against staying close to the original detection coordinates. Link weights are the calibrated conditional probabilities of being correct.

We keep division parents and daughters, gap endpoints, and interpolated nodes fixed. Refined positions are rounded to integer voxel coordinates. Changes that leave the volume or create coordinate collisions within a frame are rejected. Node counts and edges remain unchanged.

A7: Affine refinement of interpolated coordinates

We then refine the nodes inserted into gaps using the same affine motion fields, fitted from ordinary links before coordinate refinement. Starting at one endpoint, we propagate positions through the motion fields and distribute the final endpoint discrepancy linearly across the gap so that both endpoints are preserved.

Only the interpolated nodes move in this step; detected nodes, gap endpoints, and graph connections remain fixed. We again round coordinates and reject out-of-bounds positions and collisions. The submission uses both A6 and A7.

8. Submission and Results

We accelerated inference with TensorRT for the detectors and division models, mixed precision, and parallel execution on two GPUs. The matcher computes only the intermediate features it needs, and the flow fields are reused for matching and division-image alignment. Division models also reuse encoder features of original frames across overlapping temporal windows. We reuse these features in the models that take three original frames and in the original-frame branches of the parent model. The latter additionally processes aligned images.

Our final solution achieved a CV score of 0.977801 (0.540107), a public leaderboard score of 0.977 (0.52), and a private leaderboard score of 0.967 (0.47). Values in parentheses indicate Division Jaccard.

Table 1. Building the complete pipeline. Starting from A0, we progressively add or replace stages to reach the final configuration. Every row starts from the same OOF detections for all 199 training videos, with the videos, folds, and scoring implementation fixed. Optimization and post-processing can change which nodes are retained, add interpolated nodes, and refine coordinates.

ID	Configuration / change	Adjusted edge Jaccard	Division Jaccard	Total score
A0	Detection + Hungarian assignment with drift correction	0.890211	0.000000	0.890211
A1	+ Dense flow for motion compensation	0.901919	0.000000	0.901919
A2	+ Matcher candidate filtering and correspondence scores	0.901953	0.000000	0.901953
A3	+ Division models and calibration; joint link/division optimization replaces Hungarian assignment	0.911017	0.534759	0.964493
A4	+ Gap closing	0.916111	0.534759	0.969587
A5	+ Removal of components with fewer than six nodes	0.918105	0.531915	0.971297
A6	+ Affine refinement of detected coordinates	0.923141	0.540107	0.977151
A7	+ Affine refinement of interpolated coordinates: complete pipeline	0.923790	0.540107	0.977801

The total score is `Adjusted edge Jaccard + 0.1 × Division Jaccard`. A0–A2 produce no branches, so their division scores are zero. Because ordinary-link calibration also uses parent, pre, and post features, A3 introduces division modeling and graph optimization together.

The Hungarian cost is the corrected distance d in A0 and A1, and `d + 12(1 − p)` in A2, where p is the matching probability. A0–A2 retain all detections, including isolated ones. Moving from A2 to A3 also changes the candidate gate from 7 µm to the final 12 µm flow distance gate, alongside calibration, division modeling, node selection, and joint optimization.

Adding flow and matching scores to Hungarian assignment produced modest gains. The larger improvement came at A3, when division models, calibration, and joint optimization were introduced. Distances after flow compensation and matching scores also provide evidence to the downstream calibration models, which in turn guide the LP's selection of connections. The gains at A1 and A2 alone therefore do not capture their full role in the pipeline. We interpret the A3 improvement as the combined benefit of these features, division evidence, and graph constraints. Gap closing, component removal, and coordinate refinement further improved the final score.